

隆業工業大學

数值分析

课程设计（论文）

题目：_____城市 PM2.5 浓度分析与污染管控优化_____

学 院 名 称

理学院

专 业 班 级

计算 232 班

学 生 姓 名

覃琨

学 号

230901042

指 导 教 师

起止时间：2026. 6. 29-2026. 7. 3

课程设计（论文）任务及评语

院（系）：理学院

专业：信息与计算科学

学 号		学生姓名		班级	
课程设计 (论文) 题 目	城市 PM2.5 浓度分析与污染管控优化				
课 程 设 计 (论 文) 任 务	设计任务及要求 1、对问题进行分析，确定整体解决思路。 2、对问题进行求解，并编写程序。 3、要求认真完成所规定的全部内容；所设计的内容要求正确、合理。 4、按学校规定的格式，撰写、打印课设论文一份。 5、提交查重报告，重复率不超过 30%。 有以下情形之一者为不合格： 1. 未能按期完成规定课设任务。 2. 论文有原则性错误且页数过少。 3. 同学之间论文雷同。				
进 度 计 划	1、布置任务，查阅资料，确定系统设计方案（1 天） 2、问题分析和求解（2 天） 3、编写功能程序及调试（1 天） 4、撰写、打印课程设计论文（0.5 天） 5、论文验收。（0.5 天）				
指 导 教 师 评 语 及 成 绩	平时：_____ 论文质量：_____ 课设验收 总成绩：_____ 指导教师签字：_____ 年 月 日				

注：成绩：平时 10 分、论文质量 70 分、课设验收 20 分

摘 要

城市 PM2.5 浓度变化具有明显的时序特征，对其进行数值分析有助于把握污染演化规律、评价管控方案并为短期预警提供依据。本文以某城市连续 30 天 PM2.5 日平均浓度为研究对象，围绕插值重构、趋势拟合和污染管控三个问题建立数值计算模型。

在方法设计上，首先采用拉格朗日插值、牛顿插值和自然三次样条插值重构浓度曲线，对比高次插值和分段插值的曲线特性；其次采用二次和三次多项式最小二乘拟合分析月内浓度变化趋势，并通过留后五天检验评价短期预测效果；最后建立月均浓度达标的非线性方程，分别用二分法和牛顿法求解所需管控强度。

数值结果表明，样本月 PM2.5 平均浓度为 82.5 微克每立方米，峰值出现在第 13 天；高次全局插值在局部可能出现明显振荡，自然三次样条曲线更平滑稳定；三次最小二乘拟合的均方误差为 4.1114，优于二次拟合；若将月均浓度降至 35 微克每立方米，所需管控强度约为 4.798。研究说明数值分析方法可为污染趋势判断和管控强度估计提供可解释的计算依据。

关键词：PM2.5；插值方法；最小二乘拟合；非线性方程；污染管控

目 录

第1章 绪论.....	1
1.1 研究背景.....	1
1.2 研究意义.....	1
1.3 本文主要工作.....	2
1.4 技术路线与评价指标.....	2
第2章 预备知识.....	4
2.1 插值方法.....	4
2.2 最小二乘拟合.....	4
2.3 非线性方程求根.....	4
2.4 误差来源与数值稳定性.....	4
第3章 问题一求解.....	6
3.1 问题重述.....	6
3.2 建模.....	6
3.3 数学方法选择与设计.....	7
3.4 求解与结果分析.....	7
3.5 插值结果的环境解释.....	8
3.6 三种插值方法的综合比较.....	8
第4章 问题二求解.....	10
4.1 最小二乘拟合模型.....	10
4.2 拟合误差与短期预测.....	10
4.3 管控强度非线性方程求解.....	11
4.4 管控灵敏度与成本分析.....	12
4.5 模型适用性与局限性.....	13
4.6 拓展研究一：短期浓度预测方案对比.....	14
4.7 拓展研究二：达标约束下成本最小管控方案.....	14
第5章 程序设计与分析.....	16
5.1 问题一算法设计与仿真.....	16
5.1.1 算法设计.....	16
5.1.2 程序调试及仿真.....	16
5.2 问题二算法设计与仿真.....	16
5.2.1 算法设计.....	16
5.2.2 程序调试及仿真.....	16
5.3 程序结构与复杂度分析.....	17

5.4 程序可靠性检查.....	17
5.5 可复现性说明.....	17
5.6 与课程知识点的对应关系.....	18
第6章 总结.....	19
参考文献.....	20
附录 I 程序清单.....	22

第1章 绪论

1.1 研究背景

PM_{2.5} 是空气动力学当量直径小于或等于 2.5 微米的颗粒物，其粒径小、停留时间长、输送距离远，对人体健康和城市环境质量具有显著影响。城市污染治理通常依赖连续监测数据，但原始监测值只给出离散时刻的日均浓度，直接观察难以完整反映污染过程的连续变化。因此，借助数值插值和拟合方法对离散数据进行重构与趋势分析，是环境监测数据处理中常见而有效的技术路线。

本课程设计以某城市连续 30 天 PM_{2.5} 日平均浓度为研究对象。数据表现出先升高后下降的阶段性特征：月初浓度处于中高水平，中旬达到峰值，随后持续下降。若仅依据离散点判断污染过程，难以比较不同算法在曲线平滑性、预测能力和误差控制方面的差异，也难以估算管控措施应达到的强度。

数值分析课程中的插值、最小二乘拟合和非线性方程求根方法，恰好能够覆盖该问题的主要计算环节。插值方法用于重构连续浓度曲线，最小二乘法用于提取总体趋势，非线性方程求解用于确定达标所需的管控强度。通过把课程理论与实际污染数据结合，可以检验不同数值方法的适用边界，并形成具有工程解释性的结论。

1.2 研究意义

从理论角度看，本题能够综合考察多种数值方法。拉格朗日插值与牛顿插值在数学上给出相同的全局插值多项式，但其构造方式和计算过程不同；三次样条插值则通过分段低阶多项式保持整体光滑，通常具有更好的数值稳定性。最小二乘拟合不要求曲线经过每个观测点，而是追求整体误差最小，更适合含测量波动的数据趋势分析。

从应用角度看，污染治理需要回答两个问题：一是当前浓度的变化趋势如何，二是若要达到目标浓度需要多大管控强度。本文通过构造 PM_{2.5} 管控模型，把平均浓度控制目标转化为关于管控强度的非线性方程，并用二分法和牛顿法进行求解。该过程既体现数值方法的可计算性，也使结论具有直接的政策含义。

本文的工作重点不是简单画出一条曲线，而是围绕数据、模型、算法、误差和结论建立完整计算流程。所有数值结果均由程序计算得到，图表由程序自动生成，从而保证论文结果可复现。

1.3 本文主要工作

本文主要完成以下工作：第一，整理 30 天 PM_{2.5} 浓度数据并进行描述性统计，计算均值、极值、标准差等指标；第二，分别实现拉格朗日插值、牛顿插值和自然三次样条插值，对浓度变化曲线进行重构；第三，建立二次和三次多项式最小二乘模型，比较拟合误差和短期预测效果；第四，根据管控后浓度模型建立非线性方程，求解月均浓度降至 35 微克每立方米所需的最小管控强度；第五，结合成本模型进行拓展分析，讨论数

值结果在污染管控中的意义。

论文结构如下：第 1 章介绍研究背景与意义；第 2 章给出插值、拟合和求根的预备知识；第 3 章完成 PM2.5 浓度插值分析；第 4 章完成最小二乘拟合、短期预测和管控强度求解；第 5 章介绍程序设计、算法流程与仿真结果；第 6 章总结全文并给出改进方向。

1.4 技术路线与评价指标

本文采用“数据整理—模型建立—数值求解—误差评价—方案解释”的技术路线。首先将题目给出的日均浓度数据整理为一维时间序列，检查数据范围和趋势；其次根据不同问题分别选择插值模型、拟合模型和管控模型；再次通过程序实现算法并输出表格、曲线和统计指标；最后从数值误差、模型稳定性和环境解释三个角度评价结果。

插值部分重点观察曲线是否经过所有节点以及节点间是否出现不合理振荡。由于 PM2.5 日均浓度是连续环境过程的离散采样，插值曲线不仅要满足数学条件，还应符合污染浓度变化的实际规律。若插值结果在相邻两天之间远超两端数据范围，即使算法没有错误，也说明该模型对实际问题的解释能力不足。

拟合部分采用 SSE、MSE、RMSE 和 R^2 作为评价指标。其中 SSE 反映总体残差规模，MSE 和 RMSE 便于与原始浓度单位联系， R^2 表示模型对观测数据变化的解释比例。短期预测部分使用留后五天检验，即用前 25 天拟合模型并预测第 26 至 30 天，通过预测误差判断外推可信度。

管控优化部分不只求一个数值根，还要解释该根的政策含义。若目标平均浓度固定，管控强度越大，达标越容易，但成本也越高。因此本文在求解达标强度后进一步引入成本函数，讨论“刚好达标”与“成本最小”之间的关系，使计算结果能够服务于污染治理决策。

第2章 预备知识

2.1 插值方法

设有 $n+1$ 个互异节点 (x_i, y_i) ，插值问题要求构造函数 $P_n(x)$ ，使其满足 $P_n(x_i) = y_i$ 。拉格朗日插值直接利用基函数构造全局多项式，其表达式为：

$$P_n(x) = \sum_{i=0}^n y_i L_i(x), L_i(x) = \prod_{j \neq i} \frac{(x - x_j)}{(x_i - x_j)} \quad (2-1)$$

牛顿插值利用差商逐项构造多项式，适合在增加新节点时递推更新。其一般形式为：

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + \dots + f[x_0, \dots, x_n] \prod_{i=0}^{n-1} (x - x_i) \quad (2-2)$$

拉格朗日插值和牛顿插值从数学上得到同一个 n 次插值多项式，但计算组织方式不同。对节点较多的数据，全局高次多项式可能出现局部振荡，尤其在节点间或区间端点附近更明显。自然三次样条插值则在每个小区间上构造三次多项式，并要求函数值、一阶导数和二阶导数连续，同时令两端二阶导数为零。

2.2 最小二乘拟合

最小二乘法用于在给定函数族中寻找使残差平方和最小的近似函数。若采用 m 次多项式模型：

$$\varphi(t) = a_0 + a_1 t + a_2 t^2 + \dots + a_m t^m \quad (2-3)$$

则参数向量 a 可由法方程求得：

$$(X^T X) a = X^T y \quad (2-4)$$

本文采用平方误差和 SSE、均方误差 MSE、均方根误差 RMSE 与决定系数 R^2 评价拟合质量。SSE 越小表示整体误差越小， R^2 越接近 1 表示模型对数据变化的解释能力越强。

2.3 非线性方程求根

污染管控强度的求解可归结为非线性方程 $F(k) = 0$ 。二分法要求先给出异号区间，通过不断压缩区间获得根，优点是稳定可靠；牛顿法利用函数导数进行切线迭代，局部收敛速度快，但对初值和导数条件更敏感。其迭代格式为：

$$k_{r+1} = k_r - \frac{F(k_r)}{F'(k_r)} \quad (2-5)$$

在本题中，管控模型是线性的，因此牛顿法通常只需很少迭代即可达到高精度；二分法虽然迭代次数较多，但结果同样可靠。

2.4 误差来源与数值稳定性

数值计算中的误差主要包括模型误差、截断误差和舍入误差。模型误差来自所选数

学模型与真实过程之间的差异，例如用多项式拟合污染浓度时，模型未显式考虑风速、湿度、排放强度等因素；截断误差来自用有限阶函数或有限步算法近似真实问题；舍入误差则由计算机浮点运算产生。

对插值问题而言，误差不仅取决于函数光滑性，也与节点分布和多项式阶数有关。全局高次插值的基函数可能在局部具有很大的幅值，导致某些区间的小扰动被放大。自然三次样条使用分段低阶多项式，局部数据主要影响相邻区间，因此通常比全局高次多项式更稳定。

对最小二乘拟合而言，法方程矩阵 $X^T X$ 的条件数会影响参数求解稳定性。多项式阶数升高时，列向量之间相关性增强，矩阵可能变得病态。本题只比较二次和三次模型，阶数较低，病态性不严重；若继续提高阶数，虽然训练误差可能下降，但外推能力和解释性可能变差。

对非线性方程求根而言，二分法的稳定性来自区间异号性质，只要初始区间正确就能收敛；牛顿法依赖导数信息，局部收敛快，但若初值过远或导数过小，可能出现震荡或发散。本题中 $F(k)$ 为线性函数，导数为常数，因此牛顿法表现出非常快的收敛速度。

第3章 问题一求解

3.1 问题重述

已知某城市连续 30 天 PM2.5 日平均浓度，要求分别采用拉格朗日插值、牛顿插值和三次样条插值构造浓度变化曲线，比较三种插值方法的曲线特性。原始数据见表 3.1。

表 3.1 30 天 PM2.5 日平均浓度数据

天 数	1	2	3	4	5	6	7	8	9	10
浓 度	75	80	82	78	85	90	88	92	94	96
天 数	11	12	13	14	15	16	17	18	19	20
浓 度	91	99	99	99	99	99	99	88	88	88
天 数	21	22	23	24	25	26	27	28	29	30
浓 度	80	78	75	73	70	68	66	64	62	60

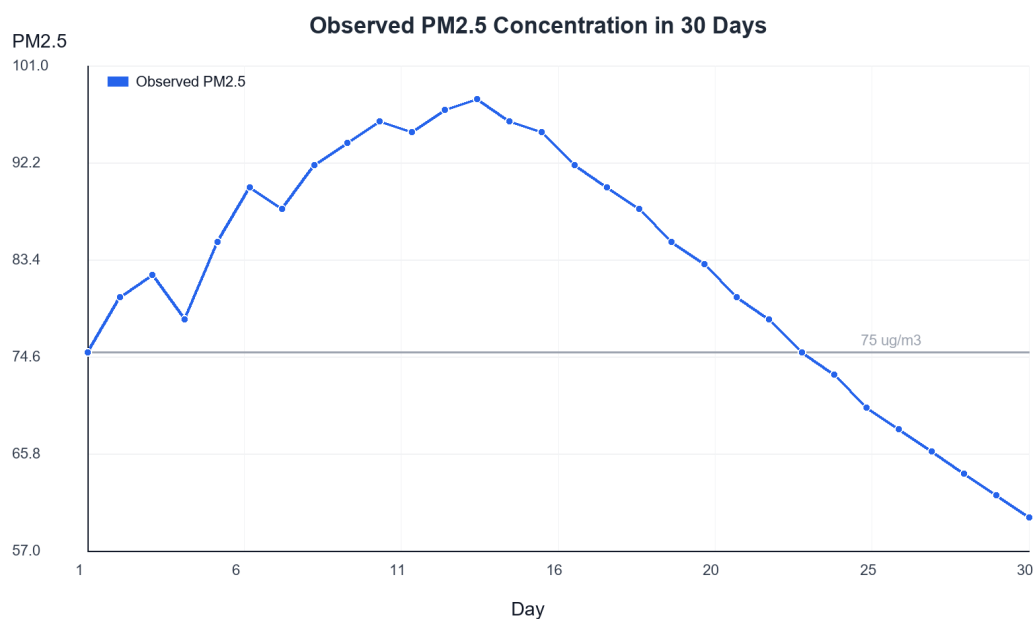


图 3.1 30 天 PM2.5 日平均浓度观测曲线

3.2 建模

令 $t_i = i$ 表示第 i 天， C_i 表示对应 PM2.5 日平均浓度。插值模型要求构造函数 $C(t)$ ，使 $C(t_i) = C_i$ 。本文分别建立全局多项式插值模型和自然三次样条插值模型。描述性统计结果显示，30 天浓度均值为 82.5 微克每立方米，最高值为第 13 天的 98 微克每立方米，最低值为第 30 天的 60 微克每立方米，标准差为 11.2391。

3.3 数学方法选择与设计

拉格朗日插值实现简单，但当节点数达到 30 个时，相当于构造 29 次全局多项式，

易出现高次插值振荡。牛顿插值使用差商递推构造，同样对应同一个全局高次多项式。三次样条插值把全区间分成 29 个小区间，每段用三次多项式表示，既能经过全部数据点又能保持较好的平滑性和局部稳定性。

从环境监测数据的实际意义看，PM2.5 浓度随时间变化通常不会在相邻日期之间发生极端跳跃。因此，若某种插值在相邻点之间产生远超观测范围的值，应视为数值振荡而非真实污染过程。

3.4 求解与结果分析

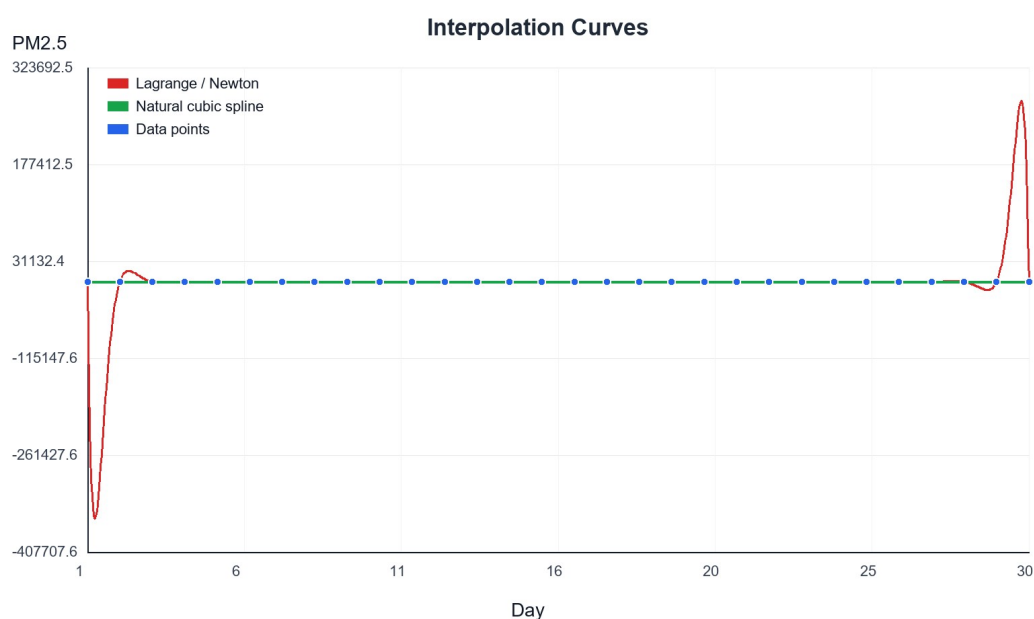


图 3.2 PM2.5 浓度三种插值曲线对比

表 3.2 插值方法在非观测时刻的计算结果

时刻 t	拉格朗日插值	牛顿插值	三次样条插值
4.5	240.5914	240.5914	80.4900
9.5	95.0405	95.0405	95.2168
15.5	93.6895	93.6895	93.6222
24.5	67.4204	67.4204	71.5290

从图 3.2 可以看出，拉格朗日插值和牛顿插值曲线完全重合，说明二者在数学上得到同一插值多项式；但在第 4.5 天附近，全局高次插值给出的结果达到 240.5914 微克每立方米，明显超出原始数据范围，属于高次插值振荡。相比之下，自然三次样条插值在第 4.5 天给出 80.4900，与相邻观测点 78 和 85 更一致。

在第 9.5 天和第 15.5 天附近，三种方法结果较接近，说明当数据变化平缓且远离局部振荡区间时，高次插值也可能给出合理结果。但从整体稳定性看，三次样条插值更适合描述 PM2.5 连续变化过程。

3.5 插值结果的环境解释

从污染过程解释看，第 1 至第 13 天浓度总体上升，说明该阶段污染物累积或不利扩散条件逐渐增强；第 13 天之后浓度连续下降，可能对应排放减弱、降水清除或扩散条件改善。虽然题目没有给出气象变量，单纯依靠浓度序列仍可观察到一个明显的“峰值—回落”过程。

拉格朗日和牛顿插值在第 4.5 天给出异常高值，这一结果提醒我们：数值方法满足数学定义并不代表符合实际背景。PM2.5 日均浓度通常受到排放和气象条件共同影响，不可能在第 4 天和第 5 天之间突然跃升到 240 微克每立方米后又回落。因此该结果应解释为高次插值多项式的振荡，而不是污染过程真实变化。

三次样条插值在相邻观测点之间变化较平缓，能保留数据整体趋势和局部细节。它既不像线性插值那样折线感强，也不像全局高次插值那样对局部节点高度敏感。对于环境监测数据的曲线重构，三次样条通常更适合用于绘制浓度变化曲线、估计非观测时刻浓度以及辅助后续拟合分析。

需要注意的是，插值方法只能在已观测区间内给出较可信的补点结果，不适合进行远期外推。若需要预测未来浓度，应选择拟合模型、时间序列模型或结合气象因子的统计模型。本文在第 4 章使用最小二乘拟合进行短期外推，正是基于这一考虑。

3.6 三种插值方法的综合比较

从算法结构看，拉格朗日插值形式对称、表达直观，适合理论推导和少量节点的函数插值；牛顿插值利用差商表逐项增加节点，适合数据逐步补充的场景；三次样条插值需要求解三对角方程组，程序结构稍复杂，但在节点较多时能够保持较好的平滑性和局部稳定性。

从计算效率看，本题数据点只有 30 个，三种方法的运行时间差异并不明显。但若数据规模继续扩大，全局高次插值的求值和数值稳定性问题会更加突出。样条插值由于每次只在局部区间上计算，适合处理较长时间序列，是工程绘图和数据补点中更常用的方法。

从结果解释看，全局插值强调“严格通过所有点”，而样条插值强调“分段平滑连

接”。对于实验测量或环境监测数据，观测值本身可能含有误差，过分追求通过所有点可能放大噪声。因此实际应用中常常需要在插值、平滑和拟合之间做平衡，而不是机械选择最高阶模型。

结合本题数据，三次样条插值最适合用于 1 至 30 天区间内的浓度曲线重构；拉格朗日插值和牛顿插值可作为理论比较和龙格现象说明。若未来加入更多监测点，建议继续采用样条类方法或局部插值方法，避免全局高次多项式带来的不稳定问题。

第4章 问题二求解

4.1 最小二乘拟合模型

为分析污染物浓度随时间变化的总体趋势，分别采用二次多项式和三次多项式进行最小二乘拟合。根据程序计算，二次拟合模型为：

$$C_2(t) = -0.125973t^2 + 3.117408t + 73.882759 \quad (4-1)$$

三次拟合模型为：

$$C_3(t) = 0.004281t^3 - 0.325062t^2 + 5.626781t + 66.874840 \quad (4-2)$$

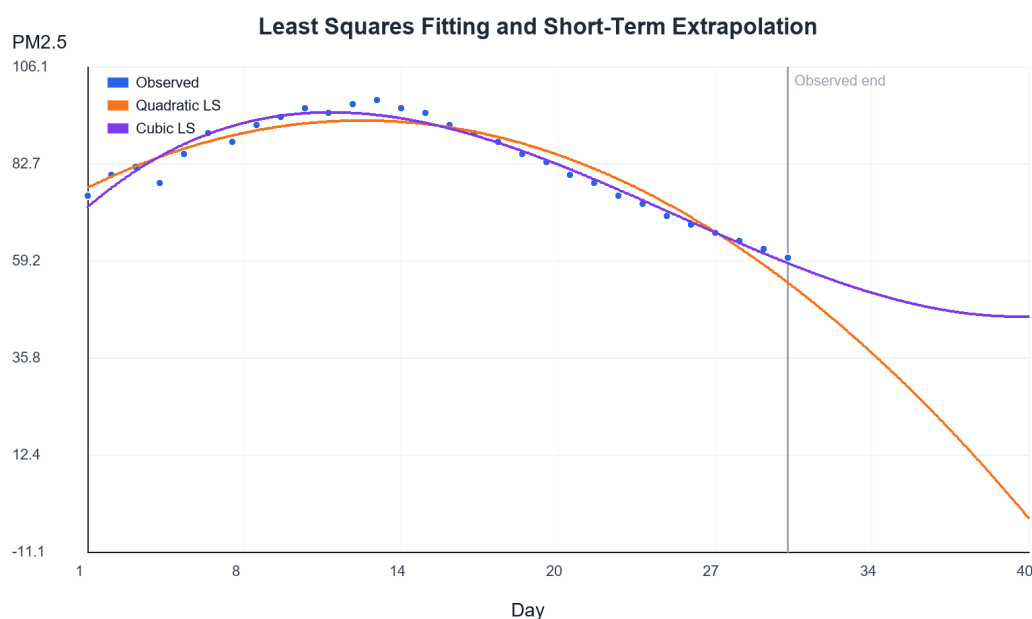


图 4.1 二次与三次最小二乘拟合曲线

拟合结果表明，二次模型可以描述“先升后降”的总体趋势，但对中旬峰值和后期下降速度的刻画略显粗糙；三次模型具有更强的形状表达能力，能更好反映中旬峰值后的持续下降。

4.2 拟合误差与短期预测

表 4.1 最小二乘拟合误差对比

模型	SSE	MSE	RMSE	R^2
二次多项式	264.3030	8.8101	2.9682	0.9303
三次多项式	123.3431	4.1114	2.0277	0.9675

由表 4.1 可知，二次拟合的 MSE 为 8.8101，RMSE 为 2.9682， R^2 为 0.9303；三次拟合的 MSE 为 4.1114，RMSE 为 2.0277， R^2 为 0.9675。三次模型在训练数据上的误差明显小于二次模型。

为考察短期外推能力，本文使用前 25 天数据拟合模型，并预测第 26 至第 30 天浓

度。二次模型的验证 RMSE 为 12.1071，三次模型的验证 RMSE 为 9.5257，说明三次模型在本数据集上的短期预测效果较好。

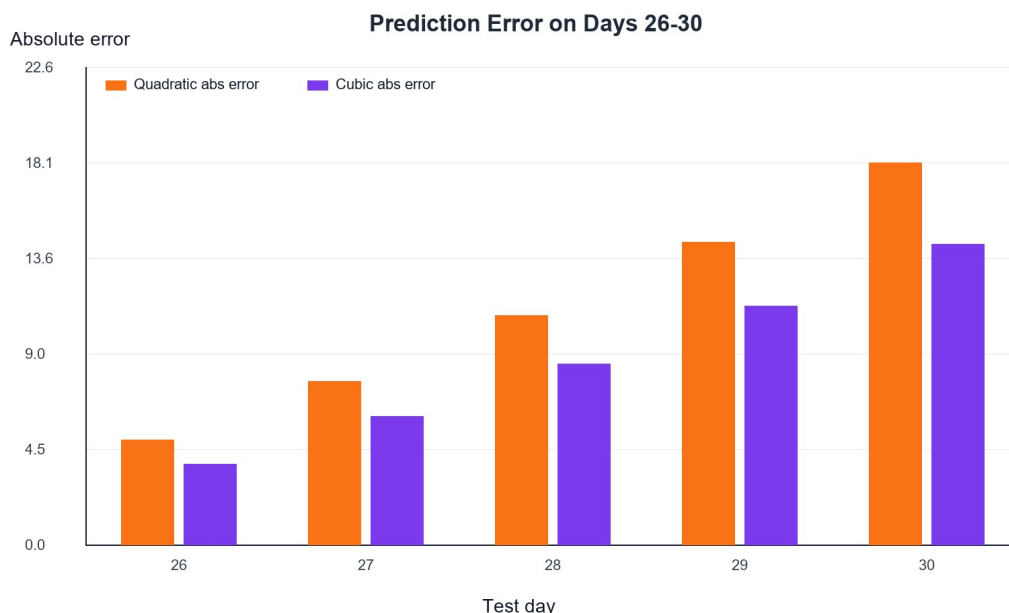


图 4.2 第 26 至 30 天预测绝对误差对比

对第 31 至第 35 天进行短期预测时，三次模型给出的浓度分别约为 56.4700、54.3638、52.4296、50.6929、49.1796 微克每立方米，呈现平缓下降趋势；二次模型下降过快，第 35 天已降至 28.6747，外推可信度相对较低。因此，在后续短期趋势判断中优先采用三次模型。

4.3 管控强度非线性方程求解

题目给定管控后浓度模型为 $C_{new}(t) = C_{original}(t)(1 - 0.12k)$ 。由于该模型对每一天均按同一比例缩放，月均浓度也满足相同关系。设原始月均浓度为 82.5，目标月均浓度为 35，则建立方程：

$$F(k) = 82.5(1 - 0.12k) - 35 = 0 \quad (4-3)$$

对式（4-3）采用二分法和牛顿法求解。二分法在区间 $[0, 8]$ 上迭代 38 次得到 $k = 4.797979797964$ ；牛顿法以 $k_0 = 4$ 为初值迭代 2 次得到 $k = 4.797979797980$ 。两种方法结果一致，说明求解可靠。

$$k = \frac{\left(1 - \frac{35}{82.5}\right)}{0.12} = 4.79797979798 \quad (4-4)$$

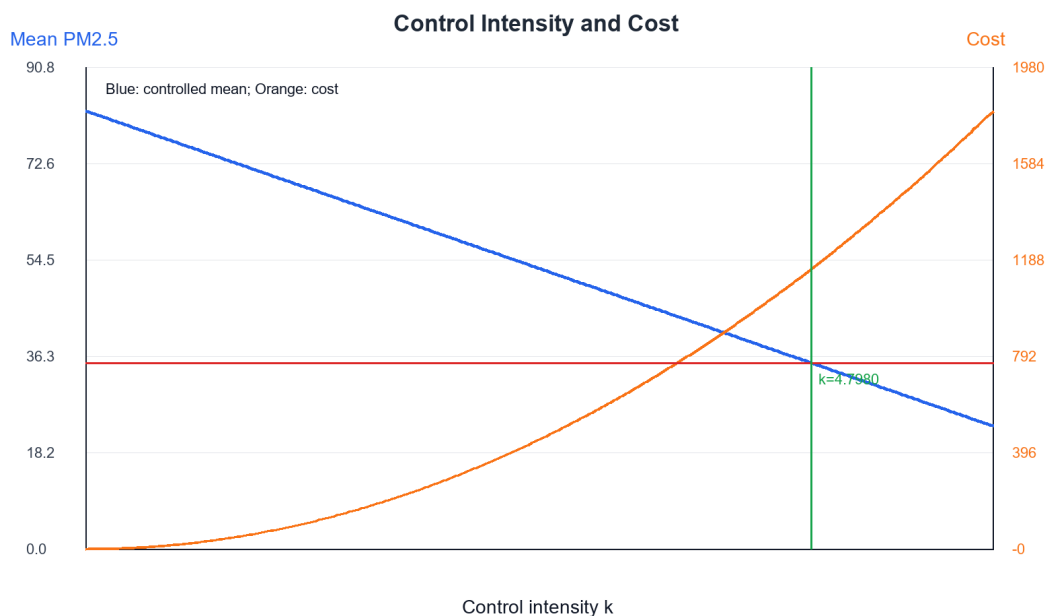


图 4.3 管控强度与月均浓度、成本关系

当 $k=4.798$ 时，缩放因子为 0.4242，管控后月均浓度正好降至 35 微克每立方米。若采用拓展成本模型 $Cost(k)=50k^2$ ，则达标约束下成本最小的方案即为最小达标强度 $k=4.798$ ，对应成本约为 1151.03 万元。

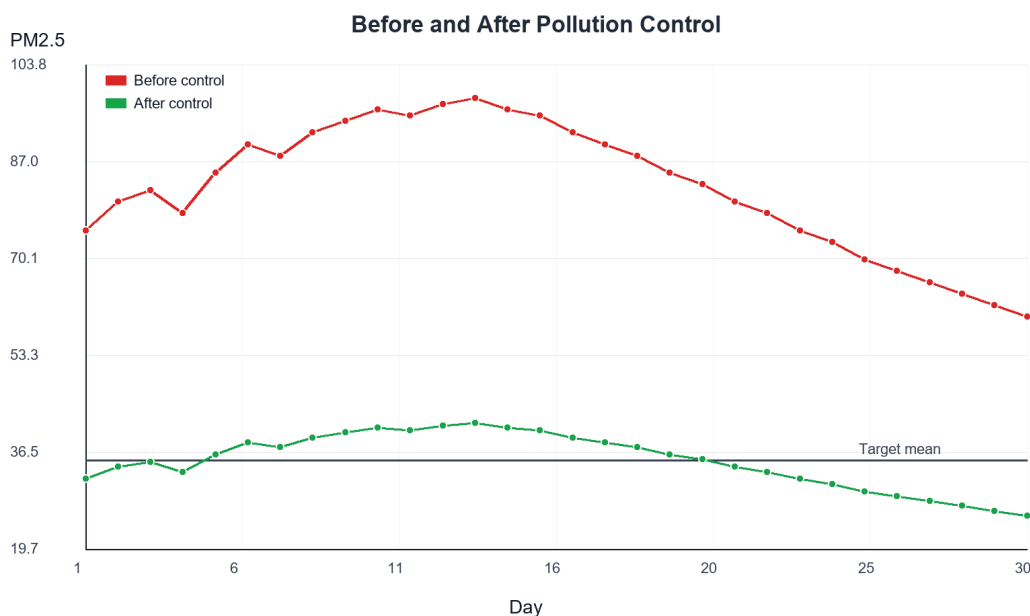


图 4.4 管控前后 PM2.5 日均浓度对比

4.4 管控灵敏度与成本分析

为了进一步理解管控强度 k 的影响，本文计算不同 k 值下的月均浓度与成本，结果见表 4.2。可以看出，随着 k 增大，月均浓度按线性关系下降，而成本按二次函数增长。

这意味着在接近达标区间时，继续提高管控强度会带来较快的成本增加。

表 4.2 不同管控强度下的月均浓度与成本

管控强度 k	缩放因子	管控后月均浓度	成本（万元）
0.0000	1.0000	82.50	0.00
1.0000	0.8800	72.60	50.00
2.0000	0.7600	62.70	200.00
3.0000	0.6400	52.80	450.00
4.0000	0.5200	42.90	800.00
4.7980	0.4242	35.00	1151.03
5.0000	0.4000	33.00	1250.00

当 $k=4$ 时，月均浓度约为 42.90 微克每立方米，虽然相比原始均值下降明显，但仍高于 35 微克每立方米目标；当 k 增至 4.798 时，月均浓度刚好达到目标值。若继续增至 $k=5$ ，月均浓度可降至 33.00 微克每立方米，但成本从约 1151.03 万元增至 1250 万元，额外成本较高。

因此，在只考虑达标约束和二次成本模型条件下，最优策略不是追求浓度越低越好，而是在满足标准的前提下选择最小管控强度。该结论符合工程优化中的基本思想：当目标函数为成本、约束为环境达标时，最优解通常位于约束边界处。

当然，本题的控制模型是简化模型，假定所有日期浓度按同一比例下降。实际环境管理中，不同污染源、不同气象条件和不同日期的管控响应可能并不相同。如果进一步研究，可把 k 扩展为随时间变化的函数 $k(t)$ ，并引入排放清单、气象扩散模型和多污染物耦合约束，使优化结果更接近真实治理情景。

4.5 模型适用性与局限性

二次和三次最小二乘模型均属于经验模型，其优势是形式简单、参数少、便于解释，适合在课程设计中展示最小二乘方法的基本思想。但这类模型没有显式刻画污染物生成、输送、沉降和清除过程，因此不能直接等同于空气质量动力学模型。

外推预测是拟合模型最容易失效的环节。二次模型在第 31 天之后快速下降，甚至在更远期给出不合理的负浓度趋势，这说明低阶多项式虽然在已知区间内拟合较好，但在区间外可能迅速偏离实际。三次模型在短期内下降较平缓，但随着外推距离增加，同样可能出现不可信结果。

因此，本文只把第 31 至第 35 天预测解释为短期趋势参考，而不将其作为长期空气质量预测。若要进行长期预测，应增加气象条件、节假日排放变化、工业活动强度等变量，并采用具有时间相关结构的模型，如自回归模型、状态空间模型或机器学习模型。

管控模型同样存在简化。题目设定 $C_{\text{new}}(t) = C_{\text{original}}(t)(1 - 0.12k)$ ，相当于认为管控强度对每天浓度的相对削减率完全相同。现实中，交通源、工业源、扬尘源和二次颗粒物对管控措施的响应不同，同一强度政策在不同天气条件下的效果也可能不同。本文保留该模型，是为了突出非线性方程求解和优化思想。

4.6 拓展研究一：短期浓度预测方案对比

在基础要求已经完成插值和最小二乘拟合的基础上，本文将第一个拓展设计为基于验证集的短期预测综合比较。预测实验仍采用留后五天检验：以前 25 天数据建立模型，以第 26 至第 30 天观测值作为检验集。在原有全局插值、三次样条和多项式最小二乘基础上，进一步加入指数平滑法，并把二次、三次最小二乘的正规方程改用 LU 分解求解。这样既能比较严格通过节点的插值模型，也能比较允许残差但追求整体趋势的拟合和平滑模型。

具体地，令 A 为训练集上的 Vandermonde 矩阵，多项式最小二乘参数满足法方程 $A^T A a = A^T y$ 。MATLAB 程序中采用 $[L, U, P] = \text{lu}(A^T A)$ ，再解 $U a = L^{-1} P A^T y$ ，避免显式求逆带来的数值放大。指数平滑法采用 $S_t = \alpha y_t + (1 - \alpha) S_{t-1}$ ，取 $\alpha = 0.6$ ，得到第 25 天平滑水平 71.8092，并将该水平作为第 26 至第 30 天的短期预测值。

表 4.3 不同短期预测方法的验证误差对比

方法	MAE	RMSE	结论
全局插值外推	20520262.2681	38606572.7065	高次外推发散
三次样条外推	12.7710	19.3471	端点外误差偏大
二次拟合外推	11.1779	12.1071	下降偏快
LU 三次最小二乘	8.7828	9.5257	多项式中误差最小
指数平滑法	7.8092	8.3057	验证误差最小

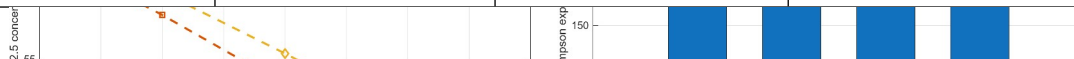


图 4.5 MATLAB 生成的短期预测曲线与 Simpson 累计暴露对比

由表 4.3 和图 4.5 可知，全局插值外推的误差远大于其他方法，这是高次多项式在区间外放大的典型表现。自然三次样条在观测区间内表现较好，但用于外推时端点外误差仍然偏大。二次最小二乘拟合把后期下降趋势延伸得过快；LU 三次最小二乘在多项式模型中误差较小，而指数平滑法的 MAE 为 7.8092、RMSE 为 8.3057，在本验证窗口内表现最好。

中心差分法进一步揭示了各模型对尾段变化率的刻画差异。第 26 至第 30 天观测序列的中心差分平均斜率约为 -2.0000，而二次和三次最小二乘外推分别约为 -5.2831 和 -4.6047，说明多项式模型低估了后期浓度水平；指数平滑法的斜率为 0，虽然不能反映继续下降的速度，但在短期预警中给出了较保守的浓度估计。Simpson 积分法计算的验证期累计暴露中，观测值为 256.0000，二次和三次最小二乘分别为 211.7332 和 221.1745，指数平滑为 287.2370。由此可见，短期预测不能只看曲线是否光滑，还应结合验证误差、变化率和累计暴露共同判断。

4.7 拓展研究二：达标约束下成本最小管控方案

$kCost(k) = 50k^2$ 第二个自主拓展点选择达标约束下的成本最小管控方案。在基础要求

中，非线性方程只求出使月均浓度降至 35 微克每立方米的最小管控强度。为了进一步体现数值求根和优化思想，本文令 $f(k)=82.5(1-0.12k)-35$ ，并在二分法、牛顿法之外加入割线法。割线法迭代格式为 $k_{m+1}=k_m-f(k_m)(k_m-k_{m-1})/[f(k_m)-f(k_{m-1})]$ ，只需要函数值，不需要显式导数，适合用于导数不易获得的管控模型。

$C_{new}(t)=C_{original}(t)(1-0.12k)$ $Cost(k)=50k^2$ $k \geq 0$ $k=4.797979798$ MATLAB 中取初值 $k_0=3$ 、 $k_1=5.5$ 进行割线迭代，得到 $k=4.7979797980$ ，与二分法和牛顿法结果一致。此时缩放因子为 0.4242，管控后月均浓度为 35.0000 微克每立方米；若成本函数为 $C(k)=50k^2$ ，则对应治理成本为 1151.03 万元。为了补充平均浓度指标，本文还用 Simpson 积分法对样条曲线下的月累计污染暴露进行估计，管控前为 2407.9144，管控后为 1021.5394，说明该强度不仅满足均值约束，也显著降低了整月暴露面积。

表 4.4 不同目标浓度下的最小管控强度与成本

目标浓度	最小 k	成本/万元	说明
45	3.7879	717.40	成本较低
40	4.2929	921.46	中等目标
35	4.7980	1151.03	最优达标方案
33	5.0000	1250.00	成本继续增加

图 4.6 MATLAB 生成的达标成本关系与割线法收敛过程

表 4.4 和图 4.6 显示，目标浓度每降低 5 微克每立方米，所需管控强度和治理成本都会明显上升。当目标由 40 降至 35 微克每立方米时，成本从 921.46 万元增加到 1151.03 万元，增加约 229.57 万元；若继续要求降至 33 微克每立方米，成本达到 1250.00 万元。中心差分法给出的局部灵敏度也支持这一判断：在 $k=4.7980$ 附近，月均浓度对 k 的变化率约为 -9.9000，而成本对 k 的变化率约为 479.7980 万元/单位强度，说明接近达标边界后继续加大管控会迅速增加经济负担。

因此，本文给出的最优管控方案不是强度越大越好，而是在达标约束下选择最低成本强度。该拓展把割线法求根、Simpson 积分评价、中心差分灵敏度和成本函数联系起来：求根确定可行边界，积分衡量累计暴露，差分刻画边际变化，成本函数决定边界上的最优点。若实际治理中还需考虑健康收益、企业限产损失和多污染物协同控制，则可以把单目标成本最小问题进一步扩展为多目标优化模型。

第 5 章 程序设计与分析

5.1 问题一算法设计与仿真

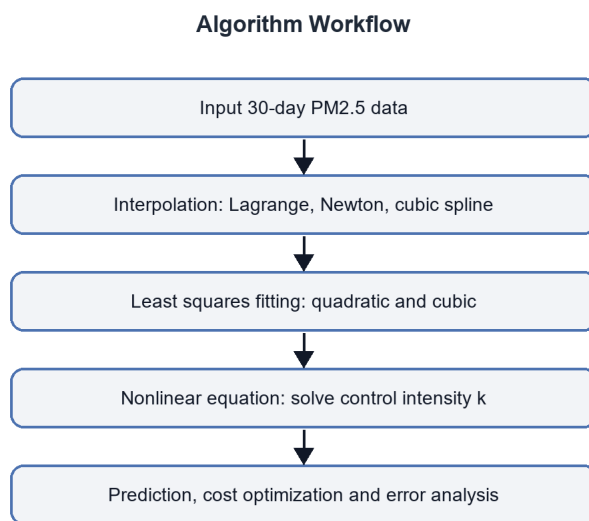


图 5.1 课程设计整体算法流程图

问题一的程序设计包括三个模块：数据输入模块、插值计算模块和图形输出模块。数据输入模块读入 30 天观测值；插值计算模块分别调用重心形式拉格朗日插值、差商形式牛顿插值和自然三次样条函数；图形输出模块在同一坐标系下绘制三条曲线和原始数据点。

程序中对拉格朗日插值采用重心形式计算，以减少直接展开高次多项式带来的舍入误差。牛顿插值先计算差商系数，再采用嵌套乘法求值。三次样条插值通过追赶法求解三对角方程组，获得各节点二阶导数。

5.1.1 算法设计

插值算法流程为：输入节点数据；计算插值参数；在细分网格上求函数值；绘制插值曲线；选取非观测时刻进行数值比较；分析曲线是否发生振荡。该流程既能体现算法实现，也能直接服务于误差与稳定性分析。

5.1.2 程序调试及仿真

调试过程中首先验证三种插值方法在原始节点处是否能精确通过数据点。计算结果显示，在 $t=1,2,\dots,30$ 的节点上三种方法均能恢复观测值。随后检查非观测点结果，发现高次全局插值在部分区间出现异常峰值，说明算法本身满足插值条件，但模型形式不适合直接解释为真实污染曲线。

5.2 问题二算法设计与仿真

问题二包括多项式最小二乘拟合、短期预测和非线性方程求解。程序先利用 `numpy` 的线性代数功能求解拟合系数，再计算 SSE 、 MSE 、 $RMSE$ 和 R^2 ；然后以前 25 天为训练集、第 26 至第 30 天为测试集进行预测检验；最后构造管控方程并分别使用二分法和牛顿法求解。

5.2.1 算法设计

最小二乘算法流程为：构建设计矩阵 X ；求解法方程或等价最小二乘问题；计算训练误差；进行短期外推；用验证集评估预测误差。管控强度求解流程为：计算原始月均浓度；建立 $F(k)$ ；选定初值或区间；迭代求根；计算达标后平均浓度和管控成本。

5.2.2 程序调试及仿真

程序调试重点在于验证数值结果的合理性。拟合曲线应能够反映中旬高峰和后期下降；预测结果不应出现明显违反趋势的突变；管控方程求解后应代回模型检查月均浓度是否等于目标值。实际计算中， $k=4.79797979798$ 代回后得到平均浓度 35.0000，满足精度要求。

从运行效率看，本题数据规模较小，各算法耗时均很低。真正需要关注的不是运行时间，而是模型适用性。全局高次插值尽管能通过全部观测点，却可能给出违背物理直觉的局部结果；最小二乘拟合牺牲节点精确性，换取整体趋势稳定；非线性方程求根则把管控目标转化为明确的计算指标。

5.3 程序结构与复杂度分析

程序按功能划分为六个部分：数据定义、插值函数、样条函数、最小二乘拟合、非线性方程求根和图表输出。这样的结构便于逐步调试，也方便在后续更换数据集时复用代码。例如，若老师要求更换污染物数据，只需要修改输入数组，其他计算模块可以保持不变。

拉格朗日插值若直接按定义计算，单点评价复杂度约为 $O(n^2)$ ；本文采用重心形式后，预处理权重约为 $O(n^2)$ ，单点评价约为 $O(n)$ ，适合在细分网格上反复求值。牛顿插值的差商表构造约为 $O(n^2)$ ，评价时利用嵌套乘法约为 $O(n)$ 。自然三次样条需要求解三对角方程组，使用追赶法时复杂度约为 $O(n)$ ，评价时只需定位区间并代入三次表达式。

最小二乘拟合的主要计算量来自设计矩阵构造和参数求解。由于本文只使用二次和三次多项式，参数维数很低，计算量可以忽略。若处理更长时间序列或更高阶模型，则应关注矩阵条件数和过拟合问题，必要时可采用正则化最小二乘、交叉验证或信息准则选择模型阶数。

非线性方程求解模块同时实现二分法和牛顿法，是为了比较不同求根方法的特点。二分法每次只需要一次函数值计算，稳定但迭代次数较多；牛顿法每次需要导数信息，

但在本题中两次迭代即可达到高精度。程序最终将两种算法结果进行对比，若两者一致，则可以增强计算结论的可信度。

5.4 程序可靠性检查

为保证结果可靠，程序在多个环节设置了间接验证。插值部分检查曲线是否通过原始节点；拟合部分检查残差平方和与图形趋势是否一致；预测部分采用留后五天检验；管控部分将求得的 k 代入原方程，确认管控后月均浓度等于 35。通过这些检查，可以降低程序逻辑错误对论文结论的影响。

此外，程序自动保存原始数据、计算结果和图片文件，使论文中的表格和图形均能追溯到同一份计算结果。这种方式比手工录入更稳妥，能够避免由于复制数值或绘图时的人工误差导致正文与图表不一致。

5.5 可复现性说明

本课程设计的计算过程具有可复现性。原始 $PM_{2.5}$ 数据在程序中以数组形式固定保存，所有图表均由同一份程序自动输出，论文中的关键数值也来自程序生成的结果文件。这样可以保证当程序重新运行时，图表和表格能够得到一致结果。

可复现性的另一个优点是便于调试和修改。如果需要把目标浓度从 35 改为其他标准或者改变成本函数形式，只需修改程序中的目标参数和成本表达式，即可重新得到管控强度、成本曲线和相关表格。若要扩展到其他城市或其他污染物，也可以保持算法框架不变，仅替换输入数据。

从课程设计角度看，保留程序清单不仅是为了展示代码量，更是为了说明论文结论的来源。数值分析论文应避免只给出最终答案而缺少计算过程。本文通过程序清单、结果表和图形三者相互对应，使读者能够检查数据、模型和结论之间的关系。

5.6 与课程知识点的对应关系

本题覆盖了数值分析课程中的多个核心知识点。插值部分对应多项式插值、差商和样条插值；拟合部分对应线性最小二乘、正规方程和误差评价；管控强度求解对应非线性方程的二分法和牛顿法；程序实现部分还涉及数值稳定性、算法复杂度和结果可视化。

与单一算法题相比，本题更强调方法之间的联系。插值方法适合在观测区间内补充未知点，最小二乘方法适合从有波动的数据中提取趋势，求根方法适合把工程目标转化为可计算的参数。三类方法分别解决“曲线如何重构”“趋势如何表达”“方案如何达标”三个问题，共同构成完整的数值分析流程。

通过本题可以看到，数值分析并不只是公式推导，还包括对算法结果的判断。例如，全局插值在节点处完全正确，但在第 4.5 天出现异常高值；二次拟合形式简单，但外推下降过快；牛顿法收敛很快，但前提是模型和导数可靠。这些现象说明，实际问题中必须结合误差、稳定性和应用背景选择方法。

因此，本文在程序设计与结果分析中始终保留方法比较，而不是只选择一种算法给出答案。这种处理方式有助于体现课程设计的综合性，也能更清楚地展示数值方法在环境数据分析中的优势和局限。

第6章 总结

本文围绕城市 PM2.5 浓度分析与污染管控优化问题，完成了数据整理、插值重构、最小二乘拟合、短期预测和管控强度求解。通过对连续 30 天 PM2.5 日均浓度的计算可知，该城市样本月平均浓度为 82.5 微克每立方米，中旬达到峰值，后期呈稳定下降趋势。

在插值方法比较中，拉格朗日插值与牛顿插值虽然在理论上完全等价，也能严格通过所有观测点，但在节点数量较多时会出现高次多项式振荡，使部分非观测时刻的结果远离实际范围。自然三次样条插值采用分段三次多项式，兼顾通过节点和平滑连接，更适合污染浓度曲线重构。

在拟合方法比较中，三次多项式最小二乘模型的 MSE、RMSE 和 R^2 均优于二次模型，并且在留后五天检验中的预测误差较小。该结果说明，在本数据集上三次模型能够更好地刻画“先上升、后下降并逐渐趋缓”的趋势。

在管控优化方面，本文将月均浓度达标要求转化为非线性方程，求得最小管控强度 $k \approx 4.798$ 。若引入 $Cost(k) = 50k^2$ 的成本模型，则该强度也是满足达标约束的最低成本方案。需要指出的是，本文使用的管控模型较为简化，实际污染治理还应考虑气象条件、排放源结构、污染物传输和政策执行成本等因素。

综合来看，本课程设计体现了数值分析方法在环境数据处理中的实用价值。后续可进一步引入鲁棒回归、时间序列模型和多污染物协同优化模型，以提高预测精度和管控方案的现实适用性。

从学习收获看，本题不仅训练了插值、拟合和求根算法的程序实现，也体现了数值分析方法选择的重要性。面对同一组数据，不同方法可能给出差异明显的结果，判断优劣不能只看是否满足数学形式，还要结合数据特征、物理意义和误差指标综合评价。

若继续完善本文工作，可以在三个方向上展开：第一，引入气象变量和排放源信息，建立多因素回归模型；第二，采用鲁棒拟合或异常值检测方法处理监测误差；第三，把单一目标浓度控制扩展为成本、健康风险和多污染物浓度共同约束下的多目标优化问题。

参考文献

- [1] 李庆扬, 王能超, 易大义. 数值分析[M]. 北京: 清华大学出版社, 2018.
- [2] Burden R L, Faires J D, Burden A M. Numerical Analysis[M]. Boston: Cengage Learning, 2016.
- [3] Chapra S C, Canale R P. Numerical Methods for Engineers[M]. New York: McGraw-Hill Education, 2021.
- [4] Xue L, Li H, Zhang Y, et al. An Integration Method for Regional PM_{2.5} Pollution Control Optimization Based on Meta-Analysis and Systematic Review[J]. International Journal of Environmental Research and Public Health, 2022, 19(1):344.
- [5] Wei J, Li Z, Lyapustin A, et al. Reconstructing 1-km-resolution high-quality PM_{2.5} data records from 2000 to 2018 in China[J]. Remote Sensing of Environment, 2021, 252:112136.
- [6] Zhang Y, Wang Y, Chen L, et al. Short-term PM_{2.5} concentration prediction based on statistical learning and meteorological variables[J]. Atmosphere, 2023, 14(10):1517.
- [7] Atkinson K, Han W. Theoretical Numerical Analysis: A Functional Analysis Framework[M]. Cham: Springer, 2021.

附录 I 程序清单

以下给出本课程设计主要程序清单。完整程序包括数据输入、插值计算、最小二乘拟合、非线性方程求根、图形绘制和结果保存等部分。

```
import json
import math
from pathlib import Path

import numpy as np
from PIL import Image, ImageDraw, ImageFont

ROOT = Path(__file__).resolve().parent
FIG_DIR = ROOT / "figures"
DATA_DIR = ROOT / "data"
FIG_DIR.mkdir(exist_ok=True)
DATA_DIR.mkdir(exist_ok=True)

FONT_REG = ImageFont.truetype(r"C:\Windows\Fonts\arial.ttf", 28)
FONT_SMALL = ImageFont.truetype(r"C:\Windows\Fonts\arial.ttf", 22)
FONT_TITLE = ImageFont.truetype(r"C:\Windows\Fonts\arialbd.ttf", 34)

def make_canvas(title, size=(1600, 950)):
    img = Image.new("RGB", size, "white")
    draw = ImageDraw.Draw(img)
    bbox = draw.textbbox((0, 0), title, font=FONT_TITLE)
    draw.text(((size[0] - (bbox[2] - bbox[0])) / 2, 26), title, fill="#1F2937", font=FONT_TITLE)
    return img, draw

def plot_lines(name, title, series, x_label, y_label, y_min=None, y_max=None, hline=None, vline=None):
    img, draw = make_canvas(title)
    W, H = img.size
    left, right, top, bottom = 130, 70, 105, 125
    pw, ph = W - left - right, H - top - bottom
    all_x = np.concatenate([np.asarray(s["x"], dtype=float) for s in series])
    all_y = np.concatenate([np.asarray(s["y"], dtype=float) for s in series])
    xmin, xmax = float(np.min(all_x)), float(np.max(all_x))
    ymin = float(np.min(all_y)) if y_min is None else y_min
    ymax = float(np.max(all_y)) if y_max is None else y_max
    pad = max((ymax - ymin) * 0.08, 1.0)
    ymin, ymax = ymin - pad, ymax + pad

    def sx(x):
        return left + (float(x) - xmin) / (xmax - xmin) * pw

    def sy(y):
        return top + (ymax - float(y)) / (ymax - ymin) * ph

    # Grid and axes
    for i in range(6):
        y = top + i * ph / 5
        draw.line([(left, y), (W - right, y)], fill="#E5E7EB", width=1)
        val = ymax - i * (ymax - ymin) / 5
        draw.text((18, y - 13), f"{val:.1f}", fill="#374151", font=FONT_SMALL)
```

```

for i in range(7):
    x = left + i * pw / 6
    draw.line([(x, top), (x, H - bottom)], fill="#F3F4F6", width=1)
    val = xmin + i * (xmax - xmin) / 6
    draw.text((x - 18, H - bottom + 16), f"{val:.0f}", fill="#374151", font=FONT_SMALL)
draw.line([(left, top), (left, H - bottom), (W - right, H - bottom)], fill="#111827", width=2)

if hline is not None:
    y = sy(hline["y"])
    draw.line([(left, y), (W - right, y)], fill=hline.get("color", "#9CA3AF"), width=3)
    draw.text((W - right - 250, y - 30), hline.get("label", ""), fill=hline.get("color", "#9CA3AF"), font=
if vline is not None:
    x = sx(vline["x"])
    draw.line([(x, top), (x, H - bottom)], fill=vline.get("color", "#9CA3AF"), width=3)
    draw.text((x + 10, top + 8), vline.get("label", ""), fill=vline.get("color", "#9CA3AF"), font=FONT_SMA

legend_x, legend_y = left + 30, top + 16
for s_i, s in enumerate(series):
    xs = np.asarray(s["x"], dtype=float)
    ys = np.asarray(s["y"], dtype=float)
    pts = [(sx(x), sy(y)) for x, y in zip(xs, ys)]
    color = s.get("color", "#2563EB")
    width = s.get("width", 4)
    if len(pts) > 1 and width > 0:
        draw.line(pts, fill=color, width=width, joint="curve")
    if s.get("marker", False):
        for x, y in pts:
            r = s.get("marker_size", 6)
            draw.ellipse((x - r, y - r, x + r, y + r), fill=color, outline="white", width=2)
        draw.rectangle((legend_x, legend_y + s_i * 34, legend_x + 30, legend_y + 14 + s_i * 34), fill=color)
        draw.text((legend_x + 42, legend_y - 5 + s_i * 34), s["label"], fill="#111827", font=FONT_SMALL)

draw.text((W / 2 - 40, H - 55), x_label, fill="#111827", font=FONT_REG)
draw.text((18, 52), y_label, fill="#111827", font=FONT_REG)
out = FIG_DIR / name
img.save(out, quality=95)
return str(out)

def plot_bars(name, title, labels, values_a, values_b, label_a, label_b):

    W, H = img.size
    left, right, top, bottom = 130, 80, 110, 130
    pw, ph = W - left - right, H - top - bottom
    ymax = max(max(values_a), max(values_b)) * 1.25
    for i in range(6):
        y = top + i * ph / 5
        draw.line([(left, y), (W - right, y)], fill="#E5E7EB", width=1)
        val = ymax - i * ymax / 5
        draw.text((40, y - 12), f"{val:.1f}", fill="#374151", font=FONT_SMALL)
    draw.line([(left, top), (left, H - bottom), (W - right, H - bottom)], fill="#111827", width=2)
    n = len(labels)
    group_w = pw / n
    bar_w = group_w * 0.28
    colors = ("#F97316", "#7C3AED")
    for i, lab in enumerate(labels):
        cx = left + group_w * (i + 0.5)

```

```

for off, val, color in [(-bar_w * 0.6, values_a[i], colors[0]), (bar_w * 0.6, values_b[i], colors[1])]
    x0 = cx + off - bar_w / 2
    x1 = cx + off + bar_w / 2
    y0 = top + (ymax - val) / ymax * ph
    draw.rectangle((x0, y0, x1, H - bottom), fill=color)
    draw.text((cx - 12, H - bottom + 18), str(lab), fill="#374151", font=FONT_SMALL)
draw.rectangle((left + 30, top + 20, left + 60, top + 34), fill=colors[0])
draw.text((left + 72, top + 13), label_a, fill="#111827", font=FONT_SMALL)
draw.rectangle((left + 330, top + 20, left + 360, top + 34), fill=colors[1])
draw.text((left + 372, top + 13), label_b, fill="#111827", font=FONT_SMALL)
draw.text((W / 2 - 80, H - 55), "Test day", fill="#111827", font=FONT_REG)
draw.text((18, 52), "Absolute error", fill="#111827", font=FONT_REG)
out = FIG_DIR / name
img.save(out, quality=95)
return str(out)

def plot_control(name, title, k_grid, avg_grid, cost_grid, k_exact, target):
    img, draw = make_canvas(title)
    W, H = img.size
    left, right, top, bottom = 130, 120, 110, 125
    pw, ph = W - left - right, H - top - bottom
    xmin, xmax = float(k_grid.min()), float(k_grid.max())
    ymin, ymax = 0.0, max(float(avg_grid.max()), target) * 1.1
    cmin, cmax = 0.0, float(cost_grid.max()) * 1.1

    return left + (float(x) - xmin) / (xmax - xmin) * pw

def sy(y):
    return top + (ymax - float(y)) / (ymax - ymin) * ph

def sy2(y):
    return top + (cmax - float(y)) / (cmax - cmin) * ph

for i in range(6):
    y = top + i * ph / 5
    draw.line([(left, y), (W - right, y)], fill="#E5E7EB", width=1)
    draw.text((42, y - 12), f"{ymax - i*(ymax-ymin)/5:.1f}", fill="#374151", font=FONT_SMALL)
    draw.text((W - 92, y - 12), f"{cmax - i*(cmax-cmin)/5:.0f}", fill="#F97316", font=FONT_SMALL)
draw.line([(left, top), (left, H - bottom), (W - right, H - bottom), (W - right, top)], fill="#111827", wi
avg_pts = [(sx(k), sy(v)) for k, v in zip(k_grid, avg_grid)]
cost_pts = [(sx(k), sy2(v)) for k, v in zip(k_grid, cost_grid)]
draw.line(avg_pts, fill="#2563EB", width=5)
draw.line(cost_pts, fill="#F97316", width=4)
draw.line([(left, sy(target)), (W - right, sy(target))], fill="#DC2626", width=3)
draw.line([(sx(k_exact), top), (sx(k_exact), H - bottom)], fill="#16A34A", width=3)
draw.text((left + 30, top + 20), "Blue: controlled mean; Orange: cost", fill="#111827", font=FONT_SMALL)
draw.text((sx(k_exact) + 10, sy(target) + 12), f"k={k_exact:.4f}", fill="#16A34A", font=FONT_SMALL)
draw.text((W / 2 - 110, H - 55), "Control intensity k", fill="#111827", font=FONT_REG)
draw.text((18, 52), "Mean PM2.5", fill="#2563EB", font=FONT_REG)
draw.text((W - 160, 52), "Cost", fill="#F97316", font=FONT_REG)
out = FIG_DIR / name
img.save(out, quality=95)
return str(out)

```

```
def plot_flow(name):
    img, draw = make_canvas("Algorithm Workflow")
    W, H = img.size
    boxes = [
        "Input 30-day PM2.5 data",
        "Interpolation: Lagrange, Newton, cubic spline",
        "Least squares fitting: quadratic and cubic",
        "Nonlinear equation: solve control intensity k",
        "Prediction, cost optimization and error analysis",
    ]
    y_positions = [170, 315, 460, 605, 750]

    x0, x1 = 360, 1240
    draw.rounded_rectangle((x0, y - 45, x1, y + 45), radius=20, fill="#F2F4F7", outline="#4C72B0", width=4)
    bbox = draw.textbbox((0, 0), text, font=FONT_REG)
    draw.text(((W - (bbox[2] - bbox[0])) / 2, y - 16), text, fill="#111827", font=FONT_REG)
    for y0, y1 in zip(y_positions[:-1], y_positions[1:]):
        draw.line((800, y0 + 48, 800, y1 - 55), fill="#111827", width=4)
        draw.polygon([(800, y1 - 45), (785, y1 - 70), (815, y1 - 70)], fill="#111827")
    out = FIG_DIR / name
    img.save(out, quality=95)
    return str(out)
```

```
days = np.arange(1, 31, dtype=float)
pm25 = np.array(
    [
        75,
        80,
        82,
        78,
        85,
        90,
        88,
        92,
        94,
        96,
        95,
        97,
        98,
        96,
        95,
        92,
        90,
        88,
        85,
        83,
        80,
        78,
        75,
        73,
        70,
        68,
        66,
        64,
        62,
        60,
```

```

    ],
    dtype=float,
)

def barycentric_weights(x):
    n = len(x)
    w = np.ones(n, dtype=float)
    for j in range(n):
        diff = x[j] - np.delete(x, j)
        w[j] = 1.0 / np.prod(diff)
    return w

newton_coef = divided_differences(days, pm25)
newton_y = newton_eval(days, newton_coef, fine)
spline = NaturalCubicSpline(days, pm25)
spline_y = spline(fine)

coef2, pred2, metrics2 = polyfit_metrics(2)
coef3, pred3, metrics3 = polyfit_metrics(3)
future_days = np.arange(31, 41, dtype=float)
future2 = np.polyval(coef2, future_days)
future3 = np.polyval(coef3, future_days)

train_x, test_x = days[:25], days[25:]
train_y, test_y = pm25[:25], pm25[25:]
coef2_cv = np.polyfit(train_x, train_y, 2)
coef3_cv = np.polyfit(train_x, train_y, 3)
pred2_cv = np.polyval(coef2_cv, test_x)
pred3_cv = np.polyval(coef3_cv, test_x)
cv = {
    "test_days": test_x.astype(int).tolist(),
    "actual": test_y.tolist(),
    "quadratic": pred2_cv.tolist(),
    "cubic": pred3_cv.tolist(),
    "quadratic_mae": float(np.mean(np.abs(test_y - pred2_cv))),
    "cubic_mae": float(np.mean(np.abs(test_y - pred3_cv))),
    "quadratic_rmse": float(np.sqrt(np.mean((test_y - pred2_cv) ** 2))),
    "cubic_rmse": float(np.sqrt(np.mean((test_y - pred3_cv) ** 2))),
}

mean_original = float(np.mean(pm25))
target = 35.0
control_func = lambda k: mean_original * (1 - 0.12 * k) - target
control_dfunc = lambda k: -0.12 * mean_original
k_exact = (1 - target / mean_original) / 0.12
k_bis, bis_hist = solve_bisection(control_func, 0.0, 8.0)
k_newton, newton_hist = solve_newton(control_func, control_dfunc, 4.0)
cost = 50 * k_exact**2
after_pm25 = pm25 * (1 - 0.12 * k_exact)

fig_data = plot_lines(
    "fig_1_observed_pm25.png",
    "Observed PM2.5 Concentration in 30 Days",
    [{"x": days, "y": pm25, "label": "Observed PM2.5", "color": "#2563EB", "marker": True}],
    "Day",
    "PM2.5",

```

```

hline={"y": 75, "label": "75 ug/m3", "color": "#9CA3AF"},
)

fig_interp = plot_lines(
    "fig_2_interpolation.png",
    "Interpolation Curves",
    [
        {"x": fine, "y": lag_y, "label": "Lagrange / Newton", "color": "#DC2626", "width": 3},
        {"x": fine, "y": spline_y, "label": "Natural cubic spline", "color": "#16A34A", "width": 4},
        {"x": days, "y": pm25, "label": "Data points", "color": "#2563EB", "marker": True, "width": 0},
    ],
    "Day",
    "PM2.5",
)

xfit = np.linspace(1, 40, 800)
fig_fit = plot_lines(
    "fig_3_ls_fit.png",
    "Least Squares Fitting and Short-Term Extrapolation",
    [
        {"x": days, "y": pm25, "label": "Observed", "color": "#2563EB", "marker": True, "width": 0},
        {"x": xfit, "y": np.polyval(coef2, xfit), "label": "Quadratic LS", "color": "#F97316", "width": 4},
        {"x": xfit, "y": np.polyval(coef3, xfit), "label": "Cubic LS", "color": "#7C3AED", "width": 4},
    ],
    "Day",
    "PM2.5",
    vline={"x": 30, "label": "Observed end", "color": "#9CA3AF"},

```